



UNIVERSIDAD SERGIO ARBOLEDA

**COLONIA DE HORMIGAS APLICADA AL PROBLEMA
DEL VIAJANTE (TSP): INSTANCIAS DE 20, 2 000 Y 200 000
CIUDADES**

Inteligencia Artificial

Mario Jiménez

Juan David Andrade

Camilo Andrés Díaz García

*Universidad Sergio Arboleda
Facultad de Sistemas e Ingeniería
Bogotá D.C., 2026*

Índice

1. Introducción	2
2. Descripción del problema y espacio de búsqueda factorial	2
3. Diseño de la solución	3
3.1. Elección de lenguaje	4
3.2. Construcción del tour	6
3.3. Número de agentes	6
3.4. Estructuras de datos y estrategias de ahorro	8
4. Implementación	10
5. Análisis de complejidad	10
6. Metodología experimental	12
7. Resultados por instancia	13
7.1. $n = 20$	13
7.2. $n = 2\,000$	14
7.3. $n = 200\,000$	15
8. Análisis de tiempo y memoria	17
8.1. Memoria: densa contra dispersa	17
8.2. Memoria por componente	17
8.3. Efecto de K y de la segunda feromona	18
8.4. Varianza entre semillas por parámetro	19
8.5. Longitud contra referencias	19
8.6. Costo de m y escalamiento con n	20
8.7. Igual presupuesto de cómputo en $n = 200\,000$	21
8.8. Aceleración y eficiencia por número de hilos	22
8.9. Tours finales	23
9. Discusión	23
10. Limitaciones	24
11. Conclusiones	25
12. Referencias	26
13. Anexo	26
13.1. Cómo reproducir cada tabla y figura	26
13.2. Glosario corto	27

1 Introducción

El problema del viajante (*Traveling Salesman Problem*, TSP) consiste en encontrar el recorrido más corto que visita una sola vez cada ciudad de un conjunto y regresa al punto de partida. Es uno de los problemas de optimización combinatoria más estudiados porque, siendo fácil de enunciar, su espacio de soluciones crece de forma factorial con el número de ciudades: para instancias grandes ninguna computadora actual puede revisar todos los recorridos posibles, y hace falta una estrategia distinta a la fuerza bruta.

Este trabajo, del curso de Inteligencia Artificial (Universidad Sergio Arboleda, SIST5036), aplica la optimización por colonia de hormigas (*Ant Colony Optimization*, ACO) al TSP en tres instancias de tamaño muy distinto: 20, 2 000 y 200 000 ciudades. El enunciado del docente pide, en concreto, medir el desempeño en función del tiempo y de la memoria, mostrando cómo ahorrar ambos recursos, usar más de 2 000 hormigas (idealmente $m \geq n$), y sustentar cada decisión con una medición real, no con una suposición. Con 200 000 ciudades, una matriz de distancias o de feromona completa ($n \times n$) ocuparía 320 GB (ver sección 5), muy por encima de la memoria de cualquier computador personal; ese es el motivo central por el que este proyecto gira en gran medida alrededor de **cómo representar el problema sin pagar ese costo**, y no solo alrededor de encontrar tours cortos.

El equipo (Mario Jiménez, Juan David Andrade y Camilo Andrés Díaz García) documenta en este informe el diseño de la solución, las estrategias de ahorro de tiempo y memoria, la metodología experimental, los resultados por instancia y sus límites honestos.

2 Descripción del problema y espacio de búsqueda factorial

En un TSP simétrico ($d_{ij} = d_{ji}$) con origen fijo (siempre se empieza en la misma ciudad, sin pérdida de generalidad porque el tour es un ciclo), el número de recorridos distintos es

$$\frac{(n-1)!}{2}, \quad (1)$$

porque hay $(n-1)!$ formas de ordenar las $n-1$ ciudades restantes, y cada tour se cuenta dos veces (una por cada sentido de recorrido). Esta cantidad crece más rápido que cualquier exponencial en n : para $n = 20$ ya es del orden de 10^{16} (más recorridos que segundos han pasado desde el big bang, multiplicados varias veces); para $n = 2000$ y $n = 200000$ el número de dígitos se cuenta en miles y en cientos de miles, respectivamente (ver figura de crecimiento factorial, calculada con la función `lgamma` para evitar desbordar cualquier tipo numérico). Esta es la razón de fondo del ejercicio: ningún algoritmo de fuerza bruta, por rápido que sea el computador, revisa ese espacio en un tiempo razonable, así que hace

falta una metaheurística que explore una fracción pequeña del espacio y aun así encuentre soluciones buenas.

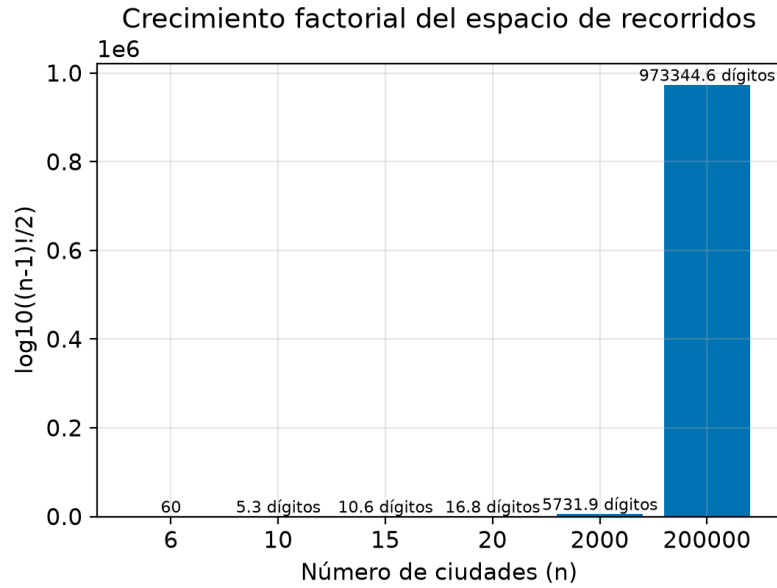


Figura 1: Crecimiento de $\log_{10}((n-1)!/2)$ para $n = 6, 10, 15, 20, 2000, 200000$.

Tabla 1: Recorridos posibles $(n-1)!/2$, calculado con lgamma.

n	$(n-1)!/2$	$\log_{10}$
6	6.000e+01	1.78
10	1.814e+05	5.26
15	4.359e+10	10.64
20	6.082e+16	16.78
2000	$> 10^{5732}$ (5732 dígitos)	5731.92
200000	$> 10^{973345}$ (973345 dígitos)	973344.55

Held-Karp resuelve el TSP de forma exacta con programación dinámica en $O(n^2 2^n)$, muchísimo mejor que $O(n!)$ pero todavía exponencial: en este trabajo solo es viable para $n = 20$ (ver sección 5), y se usa exclusivamente para verificar qué tan cerca del óptimo llega ACO en la instancia más pequeña. Para $n = 2000$ y $n = 200000$ no existe una forma práctica de calcular el óptimo exacto: la calidad se compara contra un tour por vecino más cercano (heurística ingenua, rápida de calcular) y contra la aproximación asintótica de Beardwood-Halton-Hammersley, aclarando en ambos casos que son referencias, no el óptimo.

3 Diseño de la solución

Antes de justificar cada decisión concreta de este diseño (lenguaje, número de agentes, estructuras de datos), vale explicar por qué un algoritmo como ACO funciona: se inspira

en un fenómeno biológico simple. Las hormigas reales no ven el camino más corto entre el hormiguero y una fuente de comida, pero lo encuentran de forma colectiva porque cada hormiga deja un rastro de feromona al caminar, y las hormigas siguientes tienden a preferir los caminos con más feromona. Como los caminos cortos se recorren (y se refuerzan) más veces por unidad de tiempo que los largos, el rastro de feromona en el camino corto crece más rápido; esto es **estigmergia**: coordinación indirecta entre agentes a través de cambios que cada uno deja en el ambiente, sin que haya comunicación directa entre ellos. El comportamiento colectivo (converger al camino corto) es *emergente*: ninguna hormiga individual “sabe” cuál es el camino más corto, pero el sistema completo lo encuentra. Esta idea es la que justifica, más adelante, por qué el diseño no necesita coordinar las hormigas entre sí durante la construcción (sección 3.4) ni por qué una sola capa de feromona alcanza en la mayoría de los casos (sección 3.3).

3.1 Elección de lenguaje

El proyecto se implementa en C++20 con OpenMP; no se implementó nada en Python, C ni Rust, y no se hicieron comparaciones de lenguaje. La justificación es breve y teórica, no empírica:

Python se descarta por tres razones estructurales: la sobrecarga del bucle de evaluación de bytecode del intérprete de CPython en cada operación aritmética del paso de una hormiga; el acceso irregular a memoria, que Python (incluso con *numpy* en presencia de lógica de control como la ruleta) no permite controlar con la precisión de un arreglo contiguo de $n \cdot K$ elementos; y la falta de paralelismo real dentro de un proceso, por el *Global Interpreter Lock* (GIL) de CPython, que impide que dos hilos ejecuten bytecode de Python al mismo tiempo, justo cuando la parte costosa del algoritmo (la construcción de tours por hormigas independientes) es la que se querría paralelizar con hilos.

El criterio de selección es explícito: como el ejercicio exige ahorrar memoria y tiempo, interesan los lenguajes que dan control directo sobre ambos (C, C++, Rust y Zig); se descarta el ensamblador, tanto porque el docente lo excluye como porque en un algoritmo dominado por accesos a memoria y por la lógica de la ruleta, escribirlo a mano no mejora lo que ya optimiza el compilador.

Esta es una **comparación cualitativa**: no se midió ningún otro lenguaje, y las diferencias de rendimiento esperadas entre C, C++ y Rust en este problema se esperan pequeñas (del orden de pocos puntos porcentuales), porque el tiempo lo dominan los accesos irregulares a memoria, el número de hilos disponibles y el algoritmo mismo, no el lenguaje; esto es una expectativa razonada, no un resultado medido en este trabajo.

Tabla 2: C++, Rust y C frente a los criterios relevantes para este problema.

Criterio	C++	Rust	C
Velocidad final	Equivalente a C con buen uso del compilador	Equivalente, mismo backend LLVM/GCC de bajo nivel	Equivalente, es la base de comparación
Paralelismo	OpenMP: una directiva por bucle	rayon: expresivo, pero es otro ecosistema por aprender	OpenMP igual que en C++, pero con más código manual alrededor
Estructuras de datos	Vectores, atómicos y ordenación en la librería estándar	Vectores, atómicos y ordenación en la librería estándar	A mano (arreglos y punteros crudos)
Seguridad de memoria	Depende del programador	Garantizada por el compilador (<i>borrow checker</i>)	Depende del programador
Instalación en Windows	Un solo paquete con g++ (este proyecto: WinLibs)	rustup más un enlazador, que a veces exige las herramientas de Visual Studio	Un solo paquete con gcc
Curva de aprendizaje y depuración	Conocida por el equipo, depuradores maduros	Curva más empinada (propiedad y préstamos)	Similar a C++ pero con menos abstracciones que depurar

Otras alternativas y por qué no se eligieron: **Zig** (tan rápido como C, pero un lenguaje joven, con poca documentación y comunidad pequeña); **Go** (paralelismo sencillo con goroutines, pero su recolector de basura suele dejarlo por debajo en cálculo intensivo de memoria); **Julia** (cercano a C++ en cálculo científico, pero con tiempo de compilación inicial notable y mayor uso de memoria en tiempo de ejecución); **Java o C#** (buen rendimiento después del calentamiento de la máquina virtual, pero más memoria y menos control fino, justo lo que este ejercicio pretende ahorrar); **Fortran** (excelente para cálculo con matrices densas, pero poco natural para grafos, listas de vecinas y rejillas espaciales, la estructura central de este proyecto); **Python con Numba o Cython** (el código de alto nivel sigue escrito en Python, y el docente ya advirtió que Python puede no alcanzar; usar estas herramientas exigiría además aclarar con precisión qué parte del código corre realmente compilada y cuál sigue interpretada, una complejidad que no se justifica frente a partir directamente de C++).

Se elige C++ por tres razones prácticas, no por descartar a los demás por incapacidad técnica: OpenMP se integra casi sin código adicional, el compilador se instala en Windows con un solo paquete (WinLibs), y ya existía un código base validado con pruebas automáticas. Rust habría sido una alternativa igualmente válida si el criterio principal hubiera sido la seguridad de memoria (el *borrow checker* evita en tiempo de compilación errores que en C++ solo aparecen en tiempo de ejecución); C habría sido igual de rápido, con más código para lograr lo mismo que la librería estándar de C++ da de fábrica (vectores, atómicos, ordenación). Ver docs/decisiones_diseno.md para la tabla completa de la decisión.

3.2 Construcción del tour

Esta subsección describe la parte del código (`buildTour` en `src/aco_tsp.cpp`) que cada hormiga ejecuta para armar un tour, y las fórmulas exactas que implementa. El TSP se representa como un grafo completo de n ciudades, con una distancia d_{ij} entre cada par (i, j) (euclidiana, en este trabajo). Cada arista (i, j) tiene, además de su distancia, un valor de feromona τ_{ij} que el algoritmo actualiza en cada iteración; τ_{ij} alto significa “muchas hormigas buenas pasaron por aquí en el pasado reciente”.

En cada paso de construcción, una hormiga que está en la ciudad i elige la siguiente ciudad j , entre las no visitadas, con probabilidad

$$p_{ij}^k = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{l \in N_i^k} \tau_{il}^\alpha \eta_{il}^\beta}, \quad \eta_{ij} = \frac{1}{d_{ij}}, \quad (2)$$

donde N_i^k es el conjunto de ciudades que la hormiga k aún no ha visitado (o, en la versión dispersa de este trabajo, la lista de K vecinas más cercanas de i que aún no se visitaron, ver sección 3.4), y η_{ij} es la **heurística**: entre más cerca esté una ciudad, más atractiva es, con independencia de la feromona. Los parámetros α y β controlan cuánto pesa la feromona frente a la heurística: con $\alpha = 0$ solo importa la distancia (un vecino más cercano estocástico); con $\beta = 0$ solo importa la feromona acumulada, lo que típicamente lleva a estancamiento (todas las hormigas convergen al mismo tour, así no sea bueno). La ecuación (2) no elige siempre la arista de mayor peso: el código la implementa como una **ruleta**, es decir, recorre la lista de candidatas acumulando sus pesos hasta superar un número aleatorio en $[0, \text{suma total})$; así, una arista con el doble de peso que otra tiene el doble de probabilidad de salir elegida, pero no una certeza. Esa aleatoriedad controlada es la que permite explorar en vez de repetir siempre el mismo camino, y es la razón de fondo de por qué dos corridas con distinta semilla (misma configuración) dan tours distintos, como se ve en las curvas de convergencia de la sección 7.

3.3 Número de agentes

Se adopta $m = \min(n, 20000)$: $n = 20$ con $m = 20$; $n = 2000$ con $m = 2000$; $n = 200000$ con $m = 20000$ (las 200000 ciudades se ejecutan siempre completas).

Convención de la literatura. Dorigo y Stützle (2004, *Ant Colony Optimization*, MIT Press) documentan, en la tabla de parámetros recomendados para AS y sus variantes sin búsqueda local (p. 71, Box 3.1), que el número de hormigas se fija en $m = n$; en las pp. 103–104 explican que esto hace el requisito de memoria $O(n^2)$ ($\approx 32n^2$ bytes); y en la p. 112 atribuyen la convención $m \propto n$ a los artículos originales de Dorigo, Maniezzo y

Colorni (1991, 1996). No se cita una página de esos artículos originales por no haberlos verificado directamente.

Costo y extrapolación. Con lista de candidatos de tamaño K , una iteración cuesta $O(mnK)$; con $m = n$ pasa a $O(n^2K)$. El tiempo medido en esta máquina para una iteración en $n = 200000$, $K = 8$, con la configuración ganadora (3 repeticiones cada una, resultados/costo_m_n200000_3reps.csv) fue de 0,850 s ($m = 200$), 8,681 s ($m = 2000$) y 86,940 s ($m = 20000$). El ajuste lineal por mínimos cuadrados da $t(m) \approx 0,004348m - 0,017$ segundos, con $R^2 = 1,000000$. **Como estimación por extrapolación lineal, no como corrida real**, una iteración con $m = n = 200000$ tomaría ≈ 870 s ($\approx 14,5$ min). Una medición anterior de un solo punto (Fase 2, configuración sin segunda feromona) había dado $\approx 144,85$ s para $m = 20000$ y una extrapolación de $\approx 24,3$ min; la medición con la configuración ganadora y 3 repeticiones da un tiempo por hormiga menor, consistente con lo observado directamente en la corrida real del punto 5 (≈ 85 s/iteración medidos, ver la tabla siguiente): una explicación plausible, no verificada con un experimento dedicado, es que la segunda feromona guía a las hormigas de forma más decisiva y reduce cuánto recurren a la búsqueda de reserva (la parte más cara de un paso de construcción).

Por qué el tope de 20 000. Es una decisión de presupuesto de cómputo de esta máquina (Fase 0), no una propiedad del algoritmo: diez iteraciones con $m = n = 200000$ tomarían más de 2 horas por semilla (extrapolado), mientras que con $m = 20000$ tomaron, en la corrida real, entre 13,8 y 14,2 minutos por semilla (medido directamente, docs/bitacora.md). Además, la feromona se actualiza una sola vez por iteración sin importar cuántas hormigas corrieron, así que agregar hormigas no agrega actualizaciones de feromona, solo muestras adicionales antes de esa única actualización.

Respaldo empírico. La calidad contra m en $n = 2000$ (50 iteraciones fijas, 5 semillas, resultados/barrido_m.csv) muestra rendimientos decrecientes marcados: de $m = 10$ a $m = 20000$ (2 000 veces más hormigas) la longitud media mejora solo 2,4 % (39,84 a 38,89), con un costo de cómputo casi 1 080 veces mayor.

La comparación a igual presupuesto de cómputo en $n = 200000$ ($m = 2048$ contra $m = 20000$, 15 minutos de reloj por semilla, 3 semillas; resultados/resultado_n200000_punto5.csv y resultados/resultado_n200000_punto6.csv) sí se completó:

Tabla 3: Igual presupuesto de tiempo en $n=200000$: $m=20000$ (10 iteraciones) contra $m=2048$ (~ 108 iteraciones).

Configuración	Iteraciones	L_{mejor} media	Desv. estándar	Mejora vs. NN
$m = 20000$	10	387,308	0,190	1,28 %
$m = 2048$	≈ 108	385,485	0,550	1,75 %

Con el **mismo tiempo de reloj**, $m = 2048$ (muchas más iteraciones, pocas hormigas por iteración) dio una longitud media **menor** que $m = 20000$ (pocas iteraciones, muchas hormigas), y ganó en las **3 de 3** semillas (diferencia por semilla siempre negativa: $-2,29$, $-0,99$, $-2,19$). Con solo 3 semillas la prueba de Wilcoxon no alcanza significancia convencional ($p = 0,25$, el mínimo posible con $n = 3$ pares), pero la dirección del efecto es completamente consistente. Ni la saturación de calidad contra m en $n = 2000$, ni esta comparación directa en $n = 200000$, respaldan que $m = 20000$ sea la forma más eficiente de gastar un presupuesto de tiempo fijo en esta máquina: el tope adoptado se sostiene por la convención de la literatura y por el presupuesto de tiempo *total* del proyecto, no porque los datos muestren una ventaja de calidad de usar más hormigas por iteración.

Aclaración. 20 000 es una decisión de ingeniería para esta máquina y este equipo, no un óptimo teórico. Dorigo y Stützle (2004, p. 217, sección 5.7.6) señalan que “el mejor valor de m es función del algoritmo ACO particular y de la clase de problemas atacados, y la mayoría de las veces debe fijarse experimentalmente”.

3.4 Estructuras de datos y estrategias de ahorro

La razón central del enunciado (“mostrar cómo ahorrar recursos”) se resuelve en la representación de datos, no en un truco aislado. La Tabla 4 resume las diez estrategias implementadas, cada una con su motivo, su costo antes y después, y la medición que la respalda (detalle completo en docs/estrategias_ahorro.md). La idea que las conecta a todas es evitar cualquier estructura de tamaño $O(n^2)$: en vez de una matriz completa de distancias y feromonas, cada ciudad solo conoce sus K vecinas más cercanas (encontradas por una rejilla espacial en $O(nK)$, no por comparación exhaustiva en $O(n^2)$), y esas ciudades se reenumeran por celda para que la memoria consultada por una hormiga esté físicamente cerca en el arreglo (localidad de caché). Ninguna hormiga guarda su tour completo: cada hilo reutiliza un único buffer y deposita feromona con operaciones atómicas al terminar, así que la memoria no depende de m .

Esa actualización de feromona, que en el código ocurre una sola vez por iteración (relevante para la discusión de la sección 3.3 sobre el costo de m), sigue dos pasos. Primero se evapora una fracción ρ de cada rastro:

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij}, \quad (3)$$

lo que permite “olvidar” decisiones tomadas hace muchas iteraciones y evita que la feromona crezca sin límite; esta evaporación es, precisamente, el recorrido de $O(nK)$ mencionado como parte del costo de una iteración (sección 5). Después, cada hormiga k

deposita feromona proporcional a la calidad de su tour:

$$\Delta\tau_{ij}^k = \frac{Q}{L_k} \quad \text{si la hormiga } k \text{ usó la arista } (i, j), \quad \tau_{ij} \leftarrow \tau_{ij} + \sum_k \Delta\tau_{ij}^k, \quad (4)$$

donde L_k es la longitud del tour de la hormiga k : entre más corto el tour, mayor el depósito. Q se normaliza en este trabajo como $Q = q_{fac} \cdot L_{nn}/m$ (L_{nn} : longitud de un tour por vecino más cercano), para que el depósito no dependa de la escala arbitraria de las coordenadas ni del número de hormigas; esta normalización es la que hace comparables los valores de q_{fac} usados en el barrido de parámetros (sección 6) entre instancias de tamaño distinto. Este trabajo agrega, de forma opcional, una **segunda capa de feromona** τ_2 , reforzada solo por el mejor tour global de cada iteración, con un exponente $\alpha_2 < \alpha$ para que su influencia sea menor que la de la feromona principal; el peso final de cada arista es $w_{ij} = \tau_{ij}^\alpha \tau_{2,ij}^{\alpha_2} \eta_{ij}^\beta$, que es exactamente el “peso precalculado por iteración” de la Tabla 4.

Tabla 4: Estrategias de ahorro de tiempo y memoria (detalle en docs/estrategias_ahorro.md).

Estrategia	Recurso	Antes	Después	Medición
Sin matriz $n \times n$; lista de K vecinas	Memoria	$O(n^2)$	$O(nK)$	320 GB (densa) contra 96,8 MB medidos (dispersa), $n = 200\,000$
Vecinas por rejilla espacial	Tiempo de preparación	$O(n^2)$	$O(nK)$	Preparación: 0,149 s, $n = 200\,000$
Renumeración por celda	Tiempo (caché)	Acceso disperso	Acceso contiguo	Decisión de diseño, sin comparación aislada
Búsqueda de reserva por anillos	Tiempo y memoria	Ordenar no visitadas	Solo si hace falta	Costo mayor al final del tour (ver complejidad)
Hormigas sin guardar su tour; buffer por hilo	Memoria	$O(mn)$	$O(n)$ por hilo	~ 16 GB evitados con $m = 20\,000$, $n = 200\,000$
Pesos precalculados por iteración	Tiempo	Recalcular por paso	Una vez por iteración	Cada paso: suma y ruleta de K términos
η^β precalculado	Tiempo	Recalcular por iteración	Una vez al inicio	La distancia no cambia
Paralelismo por hormiga (OpenMP)	Tiempo	Secuencial	Hasta 12 hilos	Ver aceleración y eficiencia (sección 8)
float y arreglos contiguos	Memoria y caché	double	float	Verificación en double coincide en todas las corridas
Segunda feromona solo en aristas candidatas	Memoria	$O(n^2)$ si fuera densa	$O(nK)$	Mismo argumento que la primera fila

4 Implementación

El programa de partida (`src/aco_tsp.cpp`) ya venía validado en Linux (g++ 13, OpenMP, C++20) antes de este repositorio. La Fase 1 lo portó a Windows y le añadió tres capacidades sin tocar su diseño de fondo:

- **Portabilidad:** `getrusage/ru_maxrss` (Linux) se reemplazó por `GetProcessMemoryInfo/PeakWorkingSetSize` bajo `#ifdef _WIN32`, enlazando con `-lpsapi`. El compilador elegido fue MinGW-w64 g++ 16.1.0 (WinLibs), que pasó sin problemas la prueba de C++20 + `std::atomic_ref<float>` + OpenMP (Fase 0); no hizo falta sustituir ningún primitivo del código original.
- `--input`: lectura básica de instancias TSPLIB EUC_2D (sección `NODE_COORD_SECTION`) o de un formato plano de dos columnas `x y`, con autodetección del formato. Las coordenadas originales se normalizan al cuadrado unidad guardando el factor de escala, para poder reportar longitudes en las unidades originales del archivo. El generador uniforme por semilla sigue siendo el valor por defecto; no se encontró ningún archivo de instancias del docente en el repositorio ni en la carpeta de descargas, así que las tres instancias de este informe usan ese generador.
- `--tiempo_max`: presupuesto de tiempo por iteración (distinto del presupuesto total `--time`). Al agotarse, las hormigas en curso terminan normalmente y las que no habían arrancado no se construyen; la iteración cierra igual, promediando solo sobre las hormigas que sí corrieron.

También se escribió `src/aco_denso.cpp`, una versión de referencia con matrices completas de distancia y feromona y ruleta sobre todas las ciudades no visitadas (sin lista de candidatas), que se niega a ejecutarse con $n > 5000$ y en su lugar reporta la memoria que exigiría. Sirve como línea base para medir, de forma directa, el ahorro de memoria y tiempo de la versión dispersa (sección 8).

Cuatro pruebas automáticas (`scripts/pruebas_aco.py`) verifican, en cada corrida de referencia: que el tour reportado es una permutación válida de las n ciudades; que la longitud reportada coincide con la recalculada en doble precisión; que, con la misma semilla y un solo hilo (`OMP_NUM_THREADS=1`), dos corridas idénticas dan exactamente la misma longitud; y que en $n = 20$ el resultado nunca supera al óptimo exacto de Held-Karp. Las cuatro pasan.

5 Análisis de complejidad

Cada afirmación de costo de este trabajo tiene una derivación teórica y una comprobación empírica (sección 8); aquí se presenta solo la derivación.

Espacio de búsqueda. $(n-1)!/2$ (ecuación 1); fuerza bruta inviable para cualquier n de interés práctico. Held-Karp resuelve el óptimo exacto en $O(n^2 2^n)$: para $n = 20$, $2^{20} \approx 10^6$ hace que $n^2 2^n \approx 4 \times 10^8$ operaciones sea viable en segundos; para $n = 2\,000$ o $n = 200\,000$, 2^n desborda cualquier computador existente. ACO no da esa garantía de optimalidad, pero explora una fracción minúscula del espacio en tiempo polinomial.

Construcción de una hormiga. Con lista de K candidatas, $O(nK)$: en cada uno de los n pasos del tour se recorren las K candidatas para la ruleta. A esto se suma el costo, menor en el caso habitual, de las búsquedas de reserva por rejilla cuando las K candidatas ya están visitadas; esa búsqueda de reserva tiende a ser más cara al final del tour, porque quedan pocas ciudades sin visitar y hay que revisar más anillos de celdas hasta encontrar una libre.

Una iteración. Construcción: $O(mnK)$ (m hormigas, cada una $O(nK)$). Evaporación y recálculo de pesos: $O(nK)$ (un recorrido sobre los nK arreglos de feromona/heurística). Con la convención de la literatura $m = n$ (sección 3.3), el costo de construcción de una iteración pasa a ser $O(n^2 K)$. Con matriz densa (sin lista de candidatas), estos costos serían $O(mn^2)$ en tiempo y $O(n^2)$ en memoria (dos matrices, distancia y feromona): la comparación directa entre `aco_tsp` (dispersa) y `aco_denso` (densa) en la sección 8 confirma esta diferencia de orden.

Memoria. $O(nK)$ para las estructuras de la instancia (vecinas, distancias, feromona, feromona secundaria, pesos), más $O(n)$ por hilo para el estado de construcción (buffers de tour y de visitado), independiente de m : las hormigas no guardan su tour, así que agregar más hormigas por iteración no agrega arreglos de tamaño n adicionales, solo más trabajo de CPU repitiendo el mismo buffer por hilo.

Paralelismo. Las hormigas son independientes entre sí durante la construcción (cada una solo lee la feromona y escribe en su propio buffer), así que el bucle de construcción paraleliza casi sin coordinación entre hilos (`#pragma omp for schedule(dynamic, 1)`); el depósito de feromona usa operaciones atómicas porque varias hormigas pueden depositar en la misma arista al mismo tiempo. La parte estrictamente secuencial de cada iteración es la evaporación, el recálculo de pesos y la actualización del mejor global: por la ley de Amdahl, esa fracción secuencial s acota la aceleración máxima posible con p hilos a $1/(s + (1-s)/p)$, sin importar cuántos hilos tenga el equipo. La aceleración medida contra el número de hilos (sección 8) se compara contra esta cota.

6 Metodología experimental

Máquina. Intel Core i7-8750H, 6 núcleos físicos / 12 hilos lógicos, 15,88 GB de RAM, Windows 11 Home (docs/entorno.md). Todos los tiempos de este informe se refieren a esta máquina.

Instancias y semillas. Puntos generados de forma uniforme en el cuadrado unidad, a partir de una semilla entera (no se encontró ningún archivo de instancias del docente). El barrido de parámetros usa 5 semillas por valor en $n = 2000$; la selección final de configuración y el resultado oficial de $n = 2000$ usan 8 semillas; $n = 20$ usa 5 semillas contra Held-Karp; $n = 200\,000$ usa hasta 3 semillas por el costo de cómputo (sección 3.3).

Parámetros. Configuración de partida: $K = 8$, $\alpha = 1.5$, $\beta = 5$, $\rho = 0.1$, $q_{fac} = 3$ (tomada de una nota preliminar del enunciado). El barrido de un factor a la vez (resultados/barrido_parametros.csv) y la comparación apareada con prueba de Wilcoxon (scripts/seleccion_configuracion.py) llevaron a activar la segunda capa de feromona ($\alpha_2 = 1.0$) sobre esa configuración de partida, por dar una mejora significativa ($p = 0,0078$); combinar los mejores valores individuales de cada parámetro, en cambio, no mejoró la combinación de partida ($p = 0,055$, sin diferencia significativa). Detalle completo en docs/decisiones_diseno.md.

Criterios de parada. El algoritmo se detiene por el primero de tres criterios que se cumpla: un número máximo de iteraciones, un presupuesto de tiempo total (--time), o un número de iteraciones consecutivas sin mejorar el mejor global (*stall*). Los tres se compararon en $n = 2000$, 5 semillas cada uno (resultados/barrido_criterio_parada.csv): con 50 iteraciones fijas la longitud media fue 39,01; con *stall*=10 el algoritmo corrió, en promedio, 65,2 iteraciones antes de detenerse por sí solo y llegó a una media ligeramente mejor (38,90); con un presupuesto de 5 s alcanzó apenas 38 iteraciones en promedio y quedó peor (39,31). Es decir, para esta instancia, dejar que el algoritmo decida cuándo parar (*stall*) fue tan bueno como fijar un número de iteraciones, y mejor que limitar el tiempo de forma estricta; las corridas de este informe usan, de todas formas, un número de iteraciones fijo (más fácil de comparar entre configuraciones distintas).

Métricas y su medición. Longitud del mejor tour (recalculada en doble precisión y verificada contra la reportada por el programa en cada corrida), tiempo de preparación y de ACO (std::chrono::steady_clock), y pico de memoria del proceso (GetProcessMemoryInfo/PeakWorkingSetSize en Windows). Para comparar configuraciones se usa la media, la desviación estándar, el mínimo y el máximo sobre las semillas, y una prueba de

Wilcoxon apareada (mismas semillas en ambas configuraciones) antes de afirmar que una es mejor que otra, no solo la diferencia de medias.

7 Resultados por instancia

Todas las tablas de esta sección se generan automáticamente con `scripts/generar_figuras.py` a partir de los CSV de `resultados/`; ningún número se copió a mano.

7.1 $n = 20$

5 semillas, $m = 20$, $K = 19$, contra el óptimo exacto de Held-Karp.

Tabla 5: $n=20$ contra Held-Karp, $m=20$, $K=19$, 5 semillas.

Semilla	L_{mejor}	Óptimo exacto	Gap (%)	Tour válido
1	4,154	4,106	1,176	sí
2	4,270	4,270	0,000	sí
3	3,537	3,537	0,000	sí
4	4,075	4,075	0,000	sí
5	4,266	4,266	0,000	sí

Media 0,235, desviación 0,526, mínimo 0,000, máximo 1,176 (puntos porcentuales de gap).

Cuatro de cinco semillas alcanzan el óptimo exacto; la semilla restante queda a 1,176 % del óptimo. Los cinco tours son permutaciones válidas.

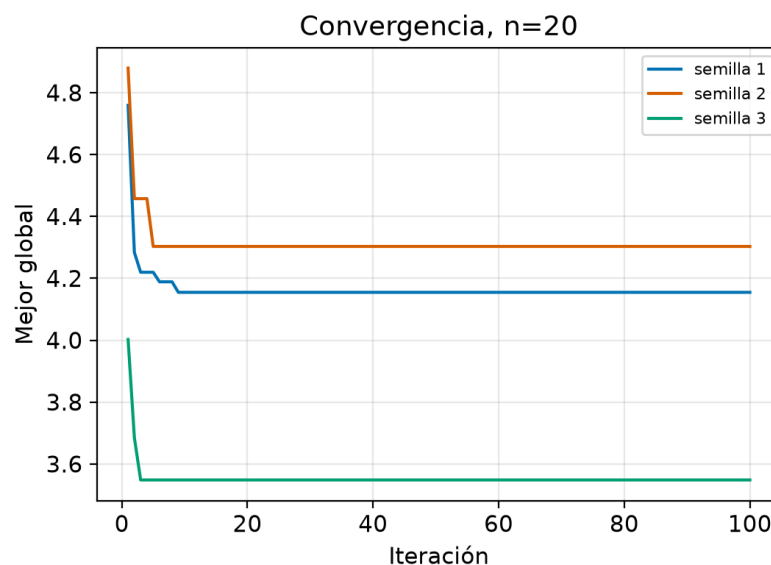


Figura 2: Convergencia (mejor global vs. iteración), $n=20$, 3 semillas.

Cómo leer esta figura: cada línea es el mejor tour encontrado hasta esa iteración, para una semilla distinta. Se espera una caída rápida en las primeras iteraciones (la feromona

todavía no distingue buenas de malas aristas) y un aplanamiento posterior (una vez que la feromona converge alrededor de las aristas del mejor tour, hay cada vez menos margen de mejora). Las tres semillas convergen a valores parecidos, coherente con que $n = 20$ es una instancia pequeña donde el algoritmo explora casi todo el espacio relevante en pocas iteraciones.

7.2 $n = 2\,000$

8 semillas, configuración ganadora ($K = 8$, $\alpha = 1.5$, $\beta = 5$, $\rho = 0.1$, $q_{fac} = 3$, segunda feromona activada), $m = 2\,000$, 50 iteraciones.

Tabla 6: $n=2\,000$, configuración ganadora, $m=2\,000$, $K=8$, 8 semillas.

Media L_{mejor}	Desv. estándar	Mínimo	Máximo
37,496	0,352	36,939	38,005

La nota preliminar del enunciado (otra máquina, una sola semilla, configuración sin segunda feromona) reportaba una longitud de 38,20 para una configuración parecida; el resultado con 8 semillas y la configuración ganadora es consistente en orden de magnitud y ligeramente mejor.

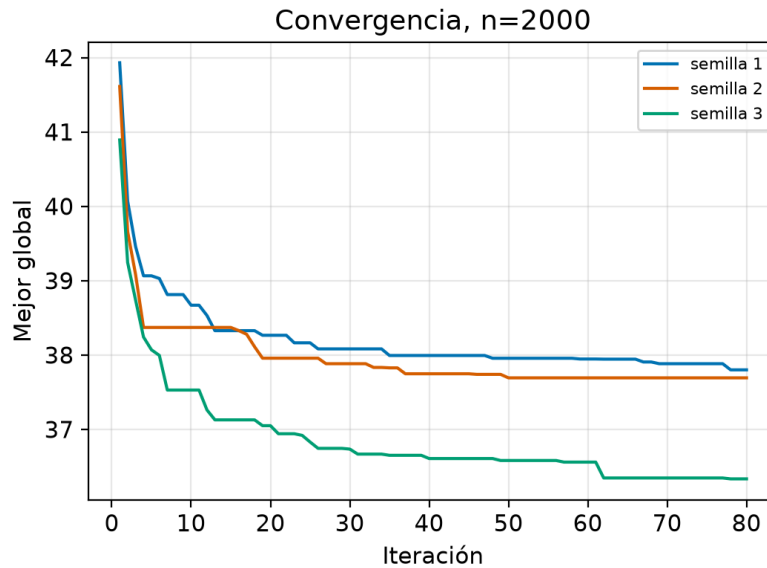


Figura 3: Convergencia (mejor global vs. iteración), $n=2\,000$, 3 semillas.

Cómo leer esta figura: igual que en $n = 20$, pero con una caída más gradual: al haber más ciudades y más pasos por tour, cada iteración aporta información más fina, y la mejora se reparte en más iteraciones antes de aplanarse.

Antes de leer los mapas de calor de α - β y ρ - Q que siguen, conviene recordar qué controla cada parámetro de la ecuación (2) y de la actualización de feromona (sección 3.4):

α , β y ρ regulan el equilibrio entre repetir lo que ya funcionó (**explotación**) y probar caminos distintos (**exploración**). α alto amplifica la feromona acumulada y favorece la explotación (riesgo: estancamiento prematuro); β alto favorece los vecinos cercanos sin importar la feromona (riesgo: ignorar información útil de iteraciones pasadas); ρ alto olvida rápido el pasado (más exploración, pero también más varianza entre corridas). Los dos mapas de calor de abajo son, precisamente, un barrido de ese equilibrio en esta instancia:

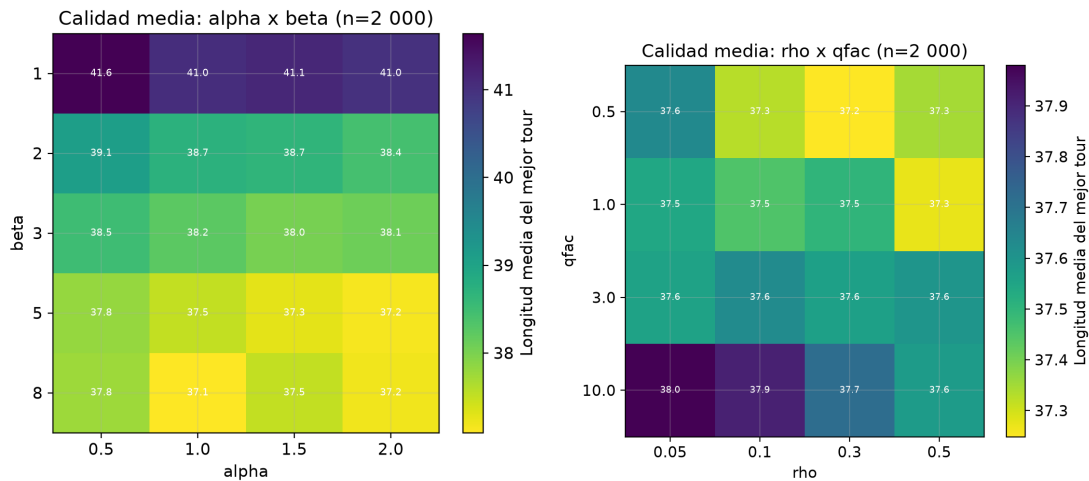


Figura 4: Superficie de calidad media, $n=2\,000$, 3 semillas por celda: $\alpha \times \beta$ (izquierda) y $\rho \times qfac$ (derecha).

Cómo leer esta figura: cada celda es la longitud media del mejor tour para esa combinación de parámetros (más oscuro/morado es peor, más claro/amarillo es mejor). El mapa alpha-beta muestra que **beta** domina: la calidad mejora de forma consistente de arriba ($\beta=1$) hacia abajo ($\beta=8$), casi sin importar la columna (alpha), mientras que alpha por sí solo cambia poco la calidad dentro de cada fila. Esto es coherente con la teoría: beta pesa la heurística (distancia), que en un TSP euclidiano es información muy confiable desde la primera iteración, mientras que alpha pesa la feromona, que recién se vuelve informativa después de varias iteraciones. El mapa rho-qfac muestra una superficie mucho más plana (rango de apenas 37,2 a 38,0): ninguno de los dos domina con la fuerza de beta, y la mejor zona (qfac bajo a moderado, rho moderado a alto) es ancha, no un único punto aislado — el algoritmo, en esta instancia, es razonablemente robusto a estos dos parámetros en concreto (observación de este trabajo, no una cita de la literatura).

7.3 $n = 200\,000$

Configuración ganadora, 3 semillas, dos corridas con el mismo presupuesto de tiempo de reloj (~15 min/semilla) pero distinto número de hormigas (detalle y análisis en la sección 8 y en docs/decisiones_diseno.md):

- $m = 20\,000$ (recomendación de la literatura, 10 iteraciones): L_{mejor} media 387,308 (desv. 0,190), mejora media del 1,28 % sobre el vecino más cercano, pico de memoria 102,4 MB.
- $m = 2\,048$ (igual presupuesto de tiempo, ≈ 108 iteraciones): L_{mejor} media 385,485 (desv. 0,550), mejora media del 1,75 % sobre el vecino más cercano, pico de memoria 102,4 MB.

Los cinco tours de ambas corridas se verificaron como permutaciones válidas. El hallazgo más relevante de esta instancia no es la calidad absoluta (ambas configuraciones quedan lejos del óptimo, esperable con tan pocas iteraciones sobre 200 000 ciudades) sino la comparación entre ellas: ver sección 8.

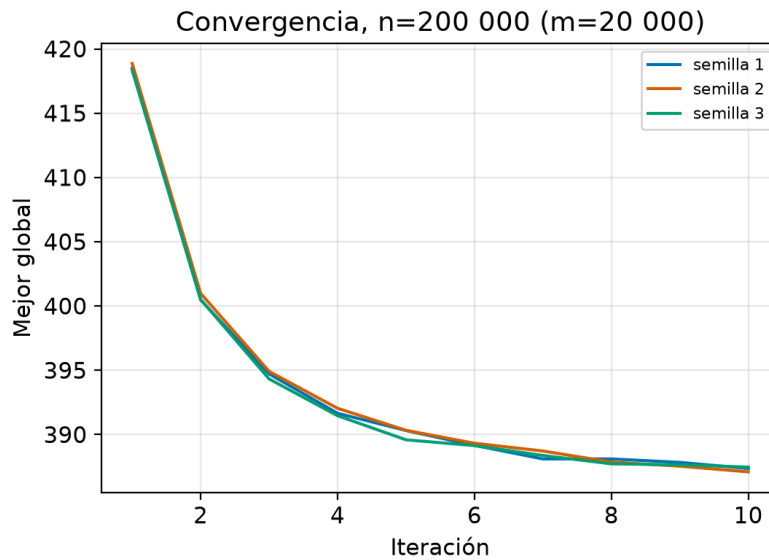


Figura 5: Convergencia (mejor global vs. iteración), n=200 000, m=20 000, 3 semillas.

Cómo leer esta figura: solo 10 iteraciones, muy pocas frente a las 50-100 usadas en $n = 20$ y $n = 2\,000$; la curva todavía cae con pendiente notable en la última iteración graficada, es decir, el algoritmo no llegó a aplanarse — coherente con la discusión de la sección 8 sobre qué tan lejos se puede llegar con el presupuesto de tiempo de esta máquina.

8 Análisis de tiempo y memoria

8.1 Memoria: densa contra dispersa

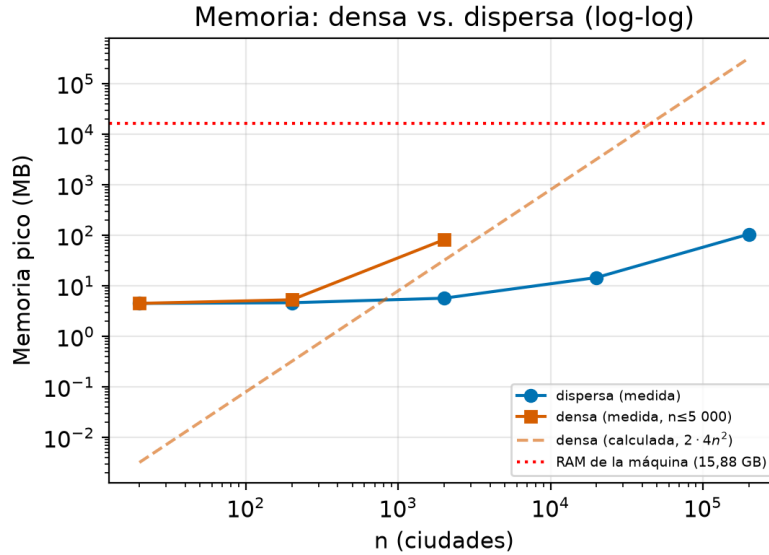


Figura 6: Memoria pico contra n , versión densa contra dispersa, escala log-log.

Cómo leer esta figura: la línea densa (medida hasta $n = 2000$, calculada más allá) crece como n^2 y cruza la RAM disponible de la máquina (15,88 GB) alrededor de $n \approx 63000$; la línea dispersa crece de forma mucho más lenta ($O(nK)$) y sigue siendo de decenas de MB incluso en $n = 200000$. Esta es la comprobación empírica directa de por qué el enunciado pide “ahorrar recursos”: sin la lista de K candidatas, la instancia más grande de este trabajo sería, sencillamente, irrealizable en esta máquina.

8.2 Memoria por componente

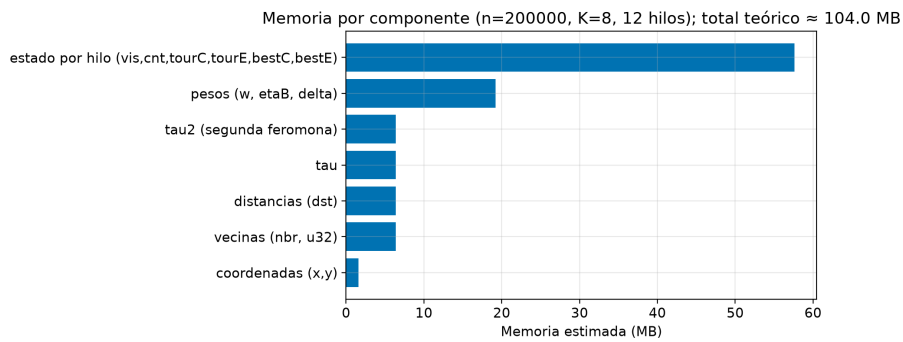


Figura 7: Memoria estimada por componente, $n=200\,000$, $K=8$, 12 hilos.

Cómo leer esta figura: cada barra es una estimación teórica directa a partir de los tamaños de los arreglos del código (coordenadas $2n$, vecinas y distancias nK cada una, feromona y feromona secundaria nK cada una, pesos $3nK$, y el estado por hilo $6n$ por cada uno de los 12 hilos). La suma teórica (104,0 MB) coincide, dentro de un 1,3 %, con el pico de memoria realmente medido en la misma instancia (105,4 MB, resultados/escalamiento_disperso.csv): esa cercanía confirma que el modelo de memoria de la sección 5 explica correctamente dónde se va cada byte, no solo el orden de magnitud total.

8.3 Efecto de K y de la segunda feromona

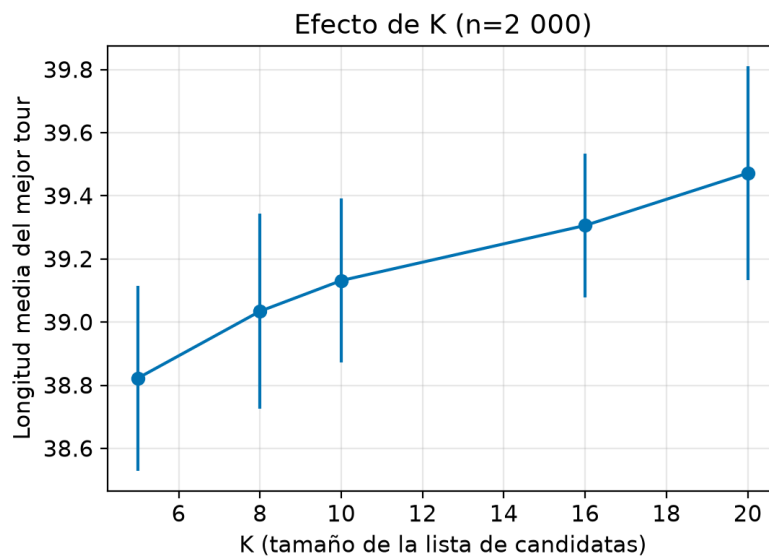


Figura 8: Efecto de K sobre la calidad, $n=2\,000$ (media $\pm$ desviación, 5 semillas).

Cómo leer esta figura: a diferencia de lo que podría esperarse (más candidatas, mejor calidad), la media empeora ligeramente al aumentar K en esta configuración base. Con $\alpha = 1$, $\beta = 3$ (valores de esta barrida, no los de la configuración ganadora), más candidatas dan a la ruleta más opciones de peso parecido entre sí, lo que diluye la guía de la heurística; esto es una observación empírica de esta configuración concreta, no una regla general sobre K .

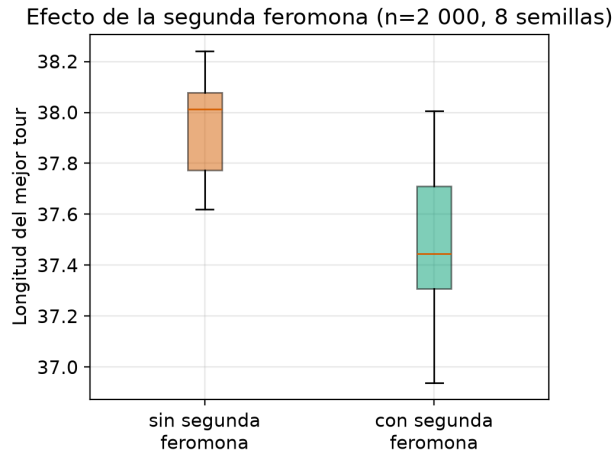


Figura 9: Efecto de la segunda feromona sobre la configuración de referencia, $n=2\,000$, 8 semillas.

Cómo leer esta figura: misma comparación apareada de la sección 3.3 (Wilcoxon, $p = 0,0078$), mostrada como diagrama de caja: la caja “con segunda feromona” está desplazada hacia longitudes menores, con una dispersión similar a la de la configuración sin ella, es decir, la mejora no viene a costa de más varianza entre semillas.

8.4 Varianza entre semillas por parámetro

Los diagramas de caja de cada parámetro del barrido (figuras/03_boxplot_*.png) muestran, para cada valor probado, la dispersión de la longitud del mejor tour entre las 5 semillas. Se incluyen en el anexo de figuras; ninguno muestra una semilla atípica que se aleje de forma anómala de las demás, lo que respalda usar la media (y no solo una corrida) como criterio de comparación entre valores de un parámetro.

8.5 Longitud contra referencias

Tabla 7: Longitud del mejor tour contra óptimo exacto ($n=20$) y aproximación BHH (n grande).

Instancia	L_{ACO}	Óptimo exacto	Aprox. BHH
$n=20$	4.060	4.051	–
$n=2\,000$	37.496	–	32.911
$n=200\,000$	387.308	–	329.109

Para $n = 2\,000$, la longitud del mejor tour (37,50) queda por encima de la aproximación asintótica de Beardwood-Halton-Hammersley (32,91), como se espera: BHH es un límite asintótico para $n \rightarrow \infty$ en el cuadrado unidad, no una cota alcanzable en una instancia finita, y además supone puntos uniformes sin borde, mientras que el cuadrado unidad sí

tiene borde (lo que sesga hacia tours algo más largos que la aproximación). Para $n = 20$, la comparación relevante es Held-Karp (óptimo exacto), no BHH (que solo es significativo para n grande).

8.6 Costo de m y escalamiento con n

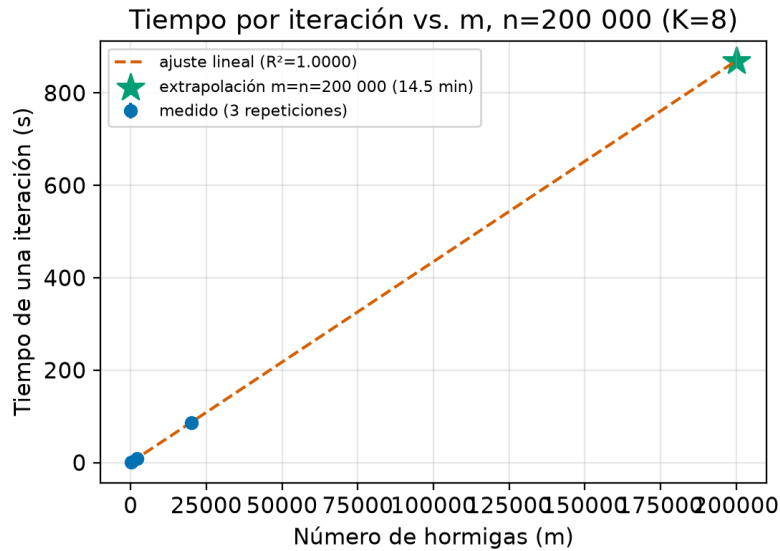


Figura 10: Tiempo de una iteración vs. m , $n=200\,000$, $K=8$, con ajuste lineal y extrapolación.

Tabla 8: Tiempo de una iteración vs. m , $n=200\,000$, $K=8$ (3 repeticiones).

m	Tiempo medio (s)	Desv. estándar (s)
200	0,850	0,015
2000	8,681	0,066
20000	86,940	0,476

Ajuste: $t(m) \approx 0,004348m - 0,017$ s, $R^2 = 1,000000$. Extrapolación a $m = n = 200000$: 869,6 s ($\approx 14,5$ min), **estimación por extrapolación, no corrida real.**

Cómo leer esta figura: los tres puntos medidos (con barras de error de 3 repeticiones) caen casi exactamente sobre la recta ajustada ($R^2 = 1,000000$), confirmando que el costo de una iteración es lineal en m a n y K fijos, tal como predice la derivación $O(mnK)$ (sección 5). La estrella marca la extrapolación a $m = n = 200000$: está fuera del rango medido, por eso se marca explícitamente como estimación y no como dato.

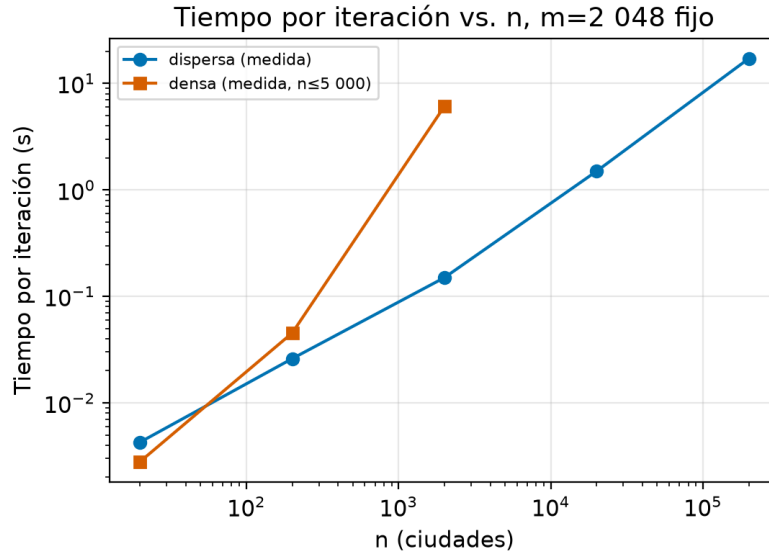


Figura 11: Tiempo por iteración vs. n , $m=2\,048$ fijo, dispersa vs. densa, log-log.

Cómo leer esta figura: en escala log-log, una relación de potencia $t \propto n^p$ se ve como una línea recta de pendiente p . La versión densa crece más rápido (mayor pendiente, consistente con $O(n^2)$ en tiempo más el costo de construir la matriz) que la dispersa ($O(nK)$, pendiente cercana a 1). Esta es la comprobación empírica, en tiempo, del mismo argumento que la Figura 6 hace en memoria.

8.7 Igual presupuesto de cómputo en $n = 200\,000$

Ver la tabla 3 y la discusión de la sección 3.3, párrafo “Respaldo empírico”: con el mismo tiempo de reloj, $m = 2048$ (más iteraciones) superó a $m = 20\,000$ (menos iteraciones) en las tres semillas probadas.

8.8 Aceleración y eficiencia por número de hilos

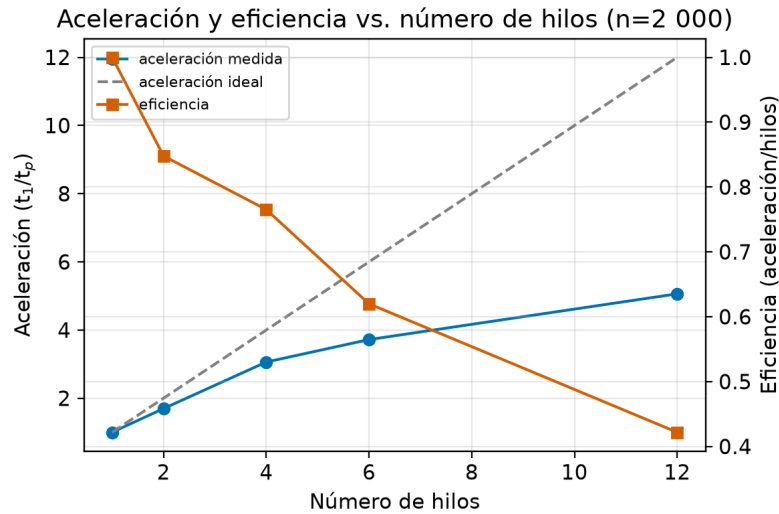


Figura 12: Aceleración y eficiencia vs. número de hilos, n=2 000, 3 semillas.

Tabla 9: Aceleración y eficiencia vs. número de hilos, n=2 000, 3 semillas.

Hilos	Tiempo medio (s)	Aceleración	Eficiencia
1	6,266	1,000	1,000
2	3,695	1,696	0,848
4	2,047	3,061	0,765
6	1,685	3,719	0,620
12	1,238	5,061	0,422

Cómo leer esta figura: la aceleración medida (t_1/t_p) queda por debajo de la ideal (t_1/p , la diagonal punteada) desde el primer punto: con 12 hilos la aceleración es de apenas $5,06 \times$ (eficiencia 42,2 %), muy lejos de $12 \times$. Dos razones, ambas consistentes con la sección 5: (1) la CPU de esta máquina tiene 6 núcleos físicos y 12 hilos lógicos (*hyperthreading*); pasar de 6 a 12 hilos comparte unidades de ejecución dentro de cada núcleo físico, así que la ganancia marginal es mucho menor que duplicar núcleos reales (la eficiencia ya cae de 62 % en 6 hilos a 42 % en 12); (2) el algoritmo es intensivo en accesos a memoria (lectura de vecinas, distancias y feromona por cada paso de cada hormiga), y con más hilos compitiendo por el mismo ancho de banda de memoria, cada hilo individual avanza más lento de lo que avanzaría solo. La parte secuencial de cada iteración (evaporación, recálculo de pesos, actualización del mejor global) explicaría, por la ley de Amdahl, una cota superior a la aceleración, pero es pequeña frente al costo de construcción; la brecha observada aquí es demasiado grande para explicarse solo por esa parte secuencial, lo que apunta al límite de ancho de banda de memoria como la causa principal.

8.9 Tours finales

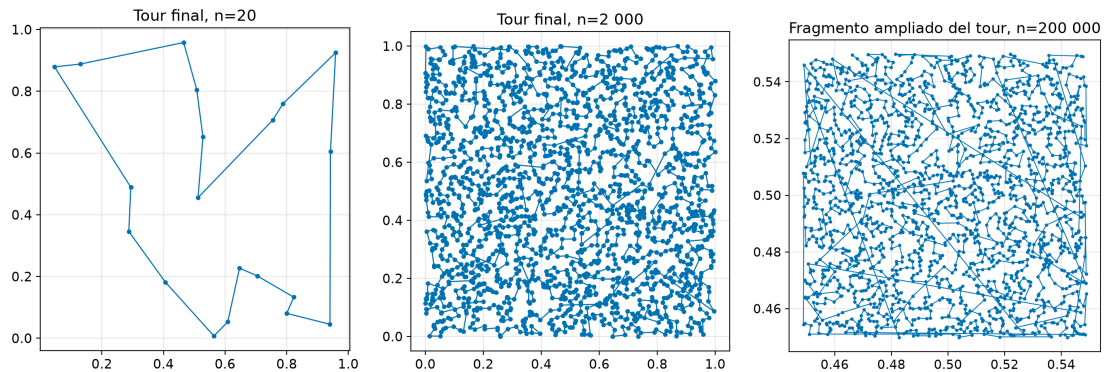


Figura 13: Tour final de $n=20$ y $n=2\,000$, y un fragmento ampliado (recorte de $\pm 0,05$ alrededor del centro) del tour de $n=200\,000$.

Cómo leer esta figura: verificación visual de que el tour es coherente (sin cruces evidentes ni saltos absurdos) para $n = 20$ y $n = 2\,000$. El fragmento de $n = 200\,000$ corresponde a una corrida corta e ilustrativa (11 iteraciones, no la corrida oficial de la sección 8), y muestra líneas largas que cruzan el recorte: con pocas iteraciones sobre una instancia tan densa, el tour todavía no termina de “localizarse” dentro de cada vecindario denso de ciudades, algo consistente con la limitación de calidad-con-pocas-iteraciones discutida en la sección de limitaciones.

9 Discusión

El resultado más claro de este trabajo es que **la representación de los datos importa más que el número de hormigas**: pasar de una matriz densa $O(n^2)$ a una lista de K vecinas $O(nK)$ es lo que hace viable siquiera intentar $n = 200\,000$ (320 GB contra unas decenas de MB medidos), mientras que subir el número de hormigas, más allá de cierto punto, da rendimientos claramente decrecientes en calidad (sección 3.3) a cambio de un costo que crece de forma lineal. La recomendación de la literatura del Ant System de usar $m = n$ hormigas tiene sentido teórico (cobertura de puntos de partida, una regla simple que escala con el problema) pero, en esta máquina y con este presupuesto de tiempo, no es practicable para $n = 200\,000$: de ahí la decisión de ingeniería de topar m en 20 000, documentada como tal y no como un óptimo universal.

El barrido de parámetros mostró además que optimizar cada parámetro por separado (manteniendo los demás en un valor por defecto) no garantiza la mejor combinación conjunta: la candidata construida así resultó, de hecho, ligeramente peor que la configuración de referencia. Lo que sí dio una mejora estadísticamente significativa fue una decisión estructural distinta: activar la segunda capa de feromona. Esto sugiere que, en trabajos

futuros con más tiempo de cómputo disponible, valdría la pena un barrido conjunto (no de un factor a la vez) alrededor de la configuración con segunda feromona, en vez de seguir optimizando parámetros individuales de la capa principal.

La comparación a igual presupuesto de tiempo en $n = 200\,000$ confirma esta lectura con un dato directo, no solo con una tendencia en $n = 2\,000$: en las tres semillas probadas, correr ≈ 108 iteraciones con $m = 2\,048$ dio una longitud menor que correr 10 iteraciones con $m = 20\,000$ en el mismo tiempo de reloj. Con solo tres semillas la diferencia no alcanza significancia estadística convencional, pero la dirección es unánime. Interpretación honesta: esto **no** demuestra que $m = 20\,000$ sea una mala elección en general, ni que la convención $m = n$ de la literatura esté equivocada (esa convención se pensó para el Ant System sin lista de candidatos y sin el tipo de restricción de tiempo de un taller universitario); lo que sí muestra es que, **en esta máquina, con esta implementación y este presupuesto de tiempo**, invertir el tiempo disponible en más iteraciones (con pocas hormigas) rindió mejor que invertirlo en más hormigas por iteración (con pocas iteraciones). Es exactamente el tipo de hallazgo empírico que este trabajo se propuso obtener: no asumir la recomendación de la literatura ni descartarla de entrada, sino medirla en las condiciones reales del equipo y reportar lo que los datos muestran, sea o no la respuesta esperada.

10 Limitaciones

- **ACO no garantiza el óptimo.** Es una metaheurística: encuentra buenas soluciones en tiempo razonable, no la mejor solución garantizada. La única instancia con verificación exacta es $n = 20$ (Held-Karp).
- **Con $n = 200\,000$ y pocas iteraciones, la mejora sobre el vecino más cercano es pequeña.** El costo de una iteración crece de forma lineal en m (sección 3.3), y la feromona se actualiza una sola vez por iteración sin importar cuántas hormigas corrieron esa iteración: con un presupuesto de tiempo fijo, hay una tensión directa entre correr más hormigas por iteración o más iteraciones, y ambas cuestan tiempo de la misma bolsa.
- **La lista de K candidatas restringe el espacio de búsqueda.** Una hormiga solo elige, en el caso habitual, entre las K vecinas más cercanas de la ciudad actual; esto acelera la construcción pero excluye, por diseño, aristas hacia ciudades lejanas que en principio podrían ser parte del tour óptimo (poco probable en un TSP euclidiano, pero no imposible).
- **Las instancias aleatorias uniformes no son instancias TSPLIB reales.** No se encontró ningún archivo de instancias del docente en el repositorio ni en la carpeta de descargas del estudiante; las tres instancias de este informe usan el generador

uniforme por semilla que ya traía el prototipo original. El programa sí soporta `--input` con TSPLIB EUC_2D (Fase 1), y se probó con instancias pequeñas sintéticas, pero ninguna figura ni tabla de este informe usa una instancia TSPLIB real.

- **Cualquier extrapolación se marca como tal.** El costo de una iteración con $m = n = 200\,000$ (sección 3.3) es una extrapolación por mínimos cuadrados sobre solo tres puntos medidos, no una corrida real; se reporta junto con su coeficiente de determinación para que el lector juzgue la confianza de esa cifra.
- **Corridas recortadas o fallidas se reportan, no se ocultan.** *(Esta sección se actualiza en el bloque de cierre con cualquier corrida real que haya fallado, se haya recortado por presupuesto de tiempo, o cuyo resultado difiera de lo esperado; ver `docs/bitacora.md` y `docs/RESUMEN_FINAL.md` para el detalle cronológico.)*

11 Conclusiones

1. El espacio de búsqueda del TSP crece de forma factorial (sección 2); para $n = 200\,000$ ninguna búsqueda exhaustiva es concebible, y el aporte central de este trabajo es una representación de datos ($O(nK)$, no $O(n^2)$) que hace viable siquiera intentar esa instancia en una computadora personal.
2. Ahorrar memoria y ahorrar tiempo, en este problema, son la misma decisión de diseño: la lista de K vecinas, la rejilla espacial y los buffers por hilo (sección 3.4) resuelven ambos a la vez, no por separado.
3. La convención de la literatura ($m = n$ para Ant System) tiene respaldo teórico verificado (sección 3.3), pero en la práctica de este equipo, con esta máquina y este tiempo disponible, no es ejecutable para $n = 200\,000$; el tope adoptado ($m = 20\,000$) es una decisión de ingeniería explícita, no una regla universal. La comparación directa a igual presupuesto de tiempo en $n = 200\,000$ (sección 8) incluso sugiere lo contrario de la intuición de “más hormigas es mejor”: con el mismo tiempo de reloj, más iteraciones con menos hormigas ($m = 2\,048$) dieron mejor calidad que menos iteraciones con más hormigas ($m = 20\,000$), en las tres semillas probadas.
4. Optimizar cada parámetro por separado no equivale a optimizar la combinación conjunta: el barrido de un factor a la vez identificó una mejora real (la segunda capa de feromona), pero también un espejismo (combinar los mejores valores individuales no mejoró el resultado).
5. La elección de C++ con OpenMP fue una decisión práctica (código base validado, paralelismo casi sin código adicional), no el resultado de descartar otros lenguajes por incapacidad: C y Rust habrían sido, en principio, igual de aptos.

Ver docs/RESUMEN_FINAL.md para el resumen ejecutivo de todo el proyecto, incluidas las corridas del bloque de cierre.

12 Referencias

- Dorigo, M., Maniezzo, V., & Coloni, A. (1996). Ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1), 29–41.
- Dorigo, M., & Stützle, T. (2004). *Ant Colony Optimization*. Cambridge, MA: MIT Press. (Verificado con página: Box 3.1, p. 71; pp. 103–104; p. 112; sección 5.7.6, p. 217; ver docs/decisiones_diseno.md para el detalle de cada cita).
- Hurbans, R. (2020). *Grokking Artificial Intelligence Algorithms*. Shelter Island, NY: Manning Publications. Capítulo 6 (Ant Colony Optimization).

13 Anexo

13.1 Cómo reproducir cada tabla y figura

Todos los comandos se ejecutan desde la raíz del repositorio (D:\camilo\Documentos\ACO_IA), con los ejecutables ya compilados (ver README.md).

- **Tabla 5 (n=20):** resultados/instancia_n20.csv, generada con 5 llamadas a aco_tsp.exe--n20--ants20--iters100--K19--exact1--seed{1..5}; tabla producida por pythonscripts/generar_figuras.pytabla_n20.
- **Tabla 6 (n=2 000):** pythonscripts/seleccion_configuracion.py (8 semillas, 3 configuraciones); tabla producida por pythonscripts/generar_figuras.pytabla_n2000.
- **Figura de crecimiento factorial:** pythonscripts/generar_figuras.pyfactorial (sin dependencias de resultados/, cálculo directo con lgamma).
- **Diagramas de caja por configuración:** pythonscripts/experimentos_fase2.pybarrido_parametros y luego pythonscripts/generar_figuras.pyboxplots.
- **Tiempo vs. m, n=200 000:** scripts/ejecutar_corridas_largas.ps1 (punto 1) y luego pythonscripts/generar_figuras.pytiempo_vs_m.
- El resto de figuras y tablas (convergencia, mapas de calor, aceleración por hilos, memoria por componente, tours finales) se generan con las funciones correspondientes de scripts/generar_figuras.py, listadas al final de ese archivo.

13.2 Glosario corto

- **Hormiga (agente):** unidad de cómputo que construye un tour completo por iteración, siguiendo la regla de transición.
- **Feromona (τ):** valor asociado a cada arista que resume, de forma indirecta, qué tan buena ha sido esa arista en tours recientes.
- **Heurística (η):** información a priori sobre el problema, en este caso el inverso de la distancia.
- **Iteración:** ciclo completo de construcción de m tours, evaporación y depósito de feromona.
- **Estigmergia:** coordinación indirecta entre agentes a través de cambios que dejan en un medio compartido (aquí, la feromona).
- **Lista de candidatas (K):** las K ciudades más cercanas a cada ciudad, usadas para restringir el costo de la construcción de $O(n)$ a $O(K)$ por paso.
- **Estancamiento:** situación en la que todas las hormigas de una iteración construyen el mismo tour (o uno casi idéntico), típicamente por una feromona demasiado concentrada en una sola ruta.